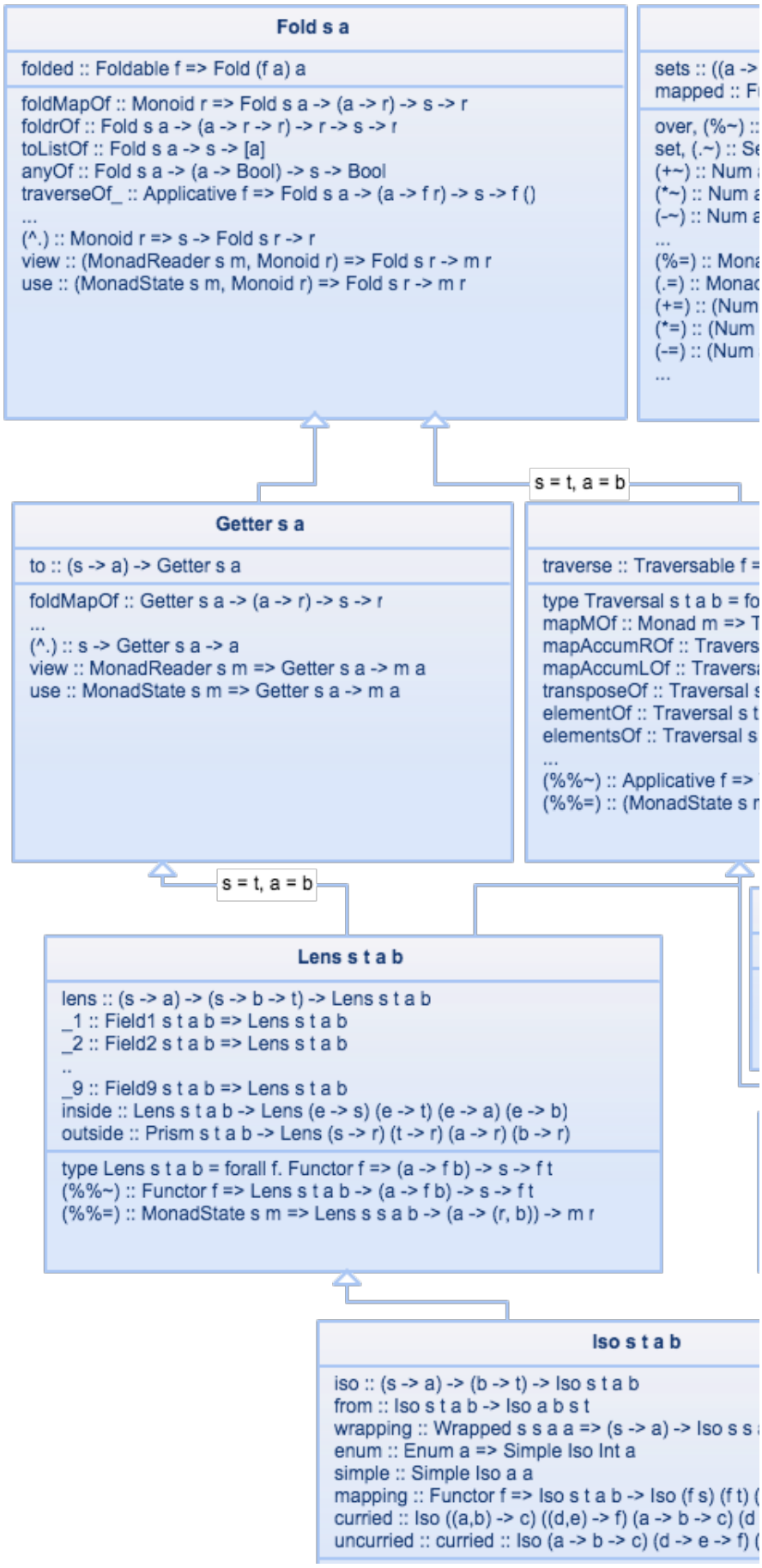
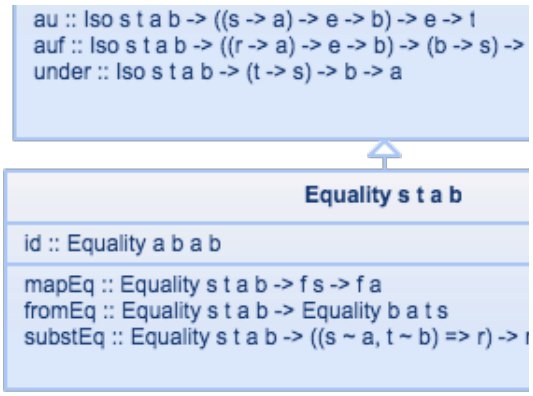
Control.Lens: https://hackage.haskell.org/package/lens

Overview





Iso ⊃ Lens ⊃ Getter ⊃ Fold
⊃ Traversal ⊃ Fold
⊃ Setter
⊃ Prism ⊃ Traversal ⊃ Fold
⊃ Setter

i.e.

```
-- Lens > Getter, Traversal, Fold, Setter
f = ...  :: Lens s t a b
f :: Lens s t a b                    -- OK
f :: Getter s t a b                  -- OK
f :: Traversal s t a b               -- OK
f :: Fold s a                        -- OK
f :: Setter s t a b                  -- OK

-- Traversal > Fold, Setter
f = ...  :: Traversal s t a b
f :: Lens s t a b                    -- ERROR
f :: Getter s t a b                  -- ERROR
f :: Traversal s t a b               -- OK
f :: Fold s a                        -- OK
f :: Setter s t a b                  -- OK

-- Getter > Fold
f = ...  :: Getter s t a b
f :: Lens s t a b                    -- ERROR
f :: Getter s t a b                  -- OK
f :: Traversal s t a b               -- ERROR
f :: Fold s a                        -- OK
f :: Setter s t a b                  -- ERROR

-- Fold
f = ...  :: Fold s a
f :: Lens s s a a                    -- ERROR
f :: Getter s s a a                  -- ERROR
f :: Traversal s s a a               -- ERROR
f :: Fold s a                        -- OK
f :: Setter s a a a                  -- ERROR

-- Setter
f = ... :: Setter s t a b
f :: Lens s t a b                    -- ERROR
```

```
f :: Getter s t a b                    -- ERROR
f :: Traversal s t a b                 -- ERROR
f :: Fold s a                          -- ERROR
f :: Setter s t a b                    -- OK
```

Modules

Control.Lens.Type

```
type Iso s t a b = forall p f. (Profunctor p, Functor f)
=> p a (f b) -> p s (f t)
type Iso' s a = Iso s s a a
type Prism s t a b = forall p f. (Choice p, Functor f)
=> p a (f b) -> p s (f t)
type Prism' s a = Prism s s a a
type Lens s t a b = forall f. Functor f => (a -> f b) ->
s -> f t
type Lens' s a = Lens s s a a
type Traversal s t a b = forall f. Applicative f => (a
-> f b) -> s -> f t
type Traversal' s a = Traversal s s a a
type Setter s t a b = forall f. Settable f => (a -> f b)
-> s -> f t
type Setter' s a = Setter s s a a
type Getter s a = forall f. (Contravariant f, Functor f)
=> (a -> f a) -> s -> f s
type Fold s a = forall f. (Contravariant f, Applicative
f) => (a -> f a) -> s -> f s

-- Indexed
type IndexedLens i s t a b b = forall f p. (Indexable i
p, Functor f) => p a (f b) -> s -> f t
type IndexedTraversal i s t a b = forall p f. (Indexable
i p, Applicative f) => p a (f b) -> s -> f t
type IndexedTraversal i s t a b = forall p f. (Indexable
i p, Settable f) => p a (f b) -> s -> f t
type IndexedGetter i s a = forall p f. (Indexable i p,
Contravariant f, Applicative f) => p a (f a) -> s -> f s
type IndexedFold i s a = forall p f. (Indexable i p,
Contravariant f, Functor f) => p a (f a) -> s -> f s



-- Abstract
type LensLike f s t a b = (a -> f b) -> s -> f t
type LensLike' f s a = Lenslike f s s a a
type Simple f s a = f s s a a

type Over p f s t a b = p a (f b) -> s -> f t
type Over' p f s a = Over p f s s a a
type Optic p f s t a b = p a (f b) -> p s (f t)
type Optic' p f s a = Optic p f s s a a
type Optical p q f s t a b = p a (f b) -> q s (f t)
type Optical' p q f s a = Optical p q f s s a a
```

Control.Lens.Lens

```
-- lens
lens :: (s -> a) -> (s -> b -> t) -> Lens s t a b
                 -- lens (getter :: s->a) (setter :: s
-> b -> t)
ilens :: (s -> (i, a)) -> (s -> b -> t) -> IndexedLens i
s t a b
iplens :: (s -> a) -> (s -> b -> t) ->
IndexPreservingLens s t a b
withLens :: ALens s t a b -> ((s -> a) -> (s -> b -> t)
-> r) -> r
```

Control.Lens.Traverse

```
-- lens
traversal :: ((a -> f b) -> s -> f t) -> LensLike f s t
a b
traverse :: Traversable f => Traversal (f a) (f b) a b
traversed :: Traversable f => IndexedTraversal Int (f a)
(f b) a b

both :: Bitraversable r => Traversal (r a a) (r b b) a b
both1 :: Bitraversable1 r => Traversal1 (r a a) (r b b)
a b
beside :: (Representable q, Applicative (Rep q),
Applicative f, Bitraversable r) => Optical p q f s t a b
-> Optical p q f s' t' a b -> Optical p q f (r s s') (r
t t') a b
taking :: (Conjoined p, Applicative f) => Int ->
Traversing p f s t a a -> Over p f s t a a
dropping :: (Conjoined p, Applicative f) => Int -> Over
p (Indexing f) s t a a -> Over p f s t a a
failing :: (Conjoined p, Applicative f) => Traversing p
f s t a b -> Over p f s t a b -> Over p f s t a b
deepOf :: (Conjoined p, Applicative f) => LensLike f s t
s t -> Traversing p f s t a b -> Over p f s t a b

partsOf :: Functor f => Traversing (->) f s t a a ->
LensLike f s t [a] [a]
partsOf' :: ATraversal s t a a -> Lens s t [a] [a]
unsafePartsOf :: Functor f => Traversing (->) f s t a b
-> LensLike f s t [a] [b]
unsafePartsOf' :: ATraversal s t a b -> Lens s t [a] [b]
holesOf :: Conjoined p => Over p (Bazaar p a a) s t a a
-> s -> [Pretext p a a t]
holes1Of :: Conjoined p => Over p (Bazaar1 p a a) s t a
a -> s -> NonEmpty (Pretext p a a t)


-- effect
```

```
traverseOf :: LensLike f s t a b -> (a -> f b) -> s -> f
t
forOf :: LensLike f s t a b -> s -> (a -> f b) -> f t
sequenceAOf :: LensLike f s t (f b) b -> s -> f t
mapMOf :: LensLike (WrappedMonad m) s t a b -> (a -> m
b) -> s -> m t
forMOf :: LensLike (WrappedMonad m) s t a b -> s -> (a
-> m b) -> m t
sequenceOf :: LensLike (WrappedMonad m) s t (m b) b -> s
-> m t
transposeOf :: LensLike ZipList s t [a] a -> s -> [t]
mapAccumLOf :: LensLike (State acc) s t a b -> (acc -> a
-> (acc, b)) -> acc -> s -> (acc, t)
mapAccumROf :: LensLike (Backwards (State acc)) s t a b
-> (acc -> a -> (acc, b)) -> acc -> s -> (acc, t)
scanr1Of :: LensLike (Backwards (State (Maybe a))) s t a
a -> (a -> a -> a) -> s -> t
scanl1Of :: LensLike (State (Maybe a)) s t a a -> (a ->
a -> a) -> s -> t
failover :: Alternative m => LensLike ((,) Any) s t a b
-> (a -> b) -> s -> m t
ifailover :: Alternative m => Over (Indexed i) ((,) Any)
s t a b -> (i -> a -> b) -> s -> m t
...

element :: Applicative t => Int -> IndexedTraversal' Int
(t a) a
elementOf :: Applicative f => LensLike (Indexing f) s t
a a -> Int -> IndexedLensLike Int f s t a a
elements :: Applicative t => (Int -> Bool)
-> IndexedTraversal' Int (t a) a
elementsOf :: Applicative f => LensLike (Indexing f) s t
a a -> (Int -> Bool) -> IndexedLensLike Int f s t a a
```

Control.Lens.Getter

```
type Getting r s a = (a -> Const r a) -> s -> Const r s

-- lens
to :: (Profunctor f, Contravariant f) => (s -> a) ->
Optic' p f s a
like :: (Profunctor p, Contravariant f, Functor f) =>
   a -> Optics' p f s a
getting ::  (Profunctor p, Profunctor q, Contravariant
f, Functor f) =>
   Optics p q f s t a b -> Optics' p q f s a



-- effect
--
-- Note: MonadReader a (->a)
view :: MonadReader s m => Getting a s a -> m a
                         --  Basic getter
```

```
                          --  e.g. view _1 (1,True) == 1

                          --       view _2 (1,True) ==
True
views :: MonadReader s m => Getting r s a -> (a -> r) ->
m r
use :: MonadState s m => Getting a s a -> m a
uses :: MonadState s m => Getting a s a -> (a -> r) -> m
r
listening :: MonadWriter w m => Getting u w u -> m a ->
m (a, u)
listenings :: MonadWriter w m => Getting u w u -> (u ->
v) -> m a -> m (a, v)


-- indexed
type IndexedGetter i s t a b = forall p f. (Indexable i
p, Contravariant f, Functor f) => p a (f b) -> s -> f t
type IndexedGetting i r s a = Indexed i a (Const r a) ->
s -> Const r s

-- lens
ito :: (Indexable i p, Contravariant f) => (s -> (i, a))
-> Over' p f s a
ilike :: (Indexable i p, Contravariant f, Functor f) =>
i -> a -> Optics' p f s a

--  effect
iview :: MonadReader s m => IndexedGetting i a s a -> m
(i, a)
iviews :: MonadReader s m => IndexedGetting i a s a ->
(i -> a -> r) -> m r
```

Control.Lens.Setter

```
type ASetter s t a b = (a -> Identity b) -> s ->
Identity t
type ASetter' s a = ASetter s s a a
type Setting p s t a b = p a (Identity b) -> s ->
Identity t
type Setting' p s a = Setting p s s a a

-- lens
sets :: (Profunctor p, Profunctor q, Settable f) => (p a
b -> q s t) -> Optical p q f s t a b
setting :: ((a -> b) -> (s -> t)) ->
IndexPreservingSetter s t a b
mapped :: Functor f => Setter (f a) (f b) a b
lifted :: Monad m => Setter (m a) (m b) a b
contramapped :: Contravariant f => Setter (f b) (f a) a
b
argument :: Profunctor p => Setter (p b r) (p a r) a b
```

```
-- effect
set :: ASetter s t a b -> b -> s -> t
over :: ASetter s t a b -> (a -> b) -> s -> t
assign :: MonadState s m => ASetter s s a b -> b -> m ()
modify :: MonadState s m => ASetter s s a b -> (a -> b)
-> m ()
...
```

Control.Lens.Fold

```
-- lens
pre :: Getting (First a) s a -> IndexPreservingGetter s
(Maybe a)
folding :: Foldable f => (s -> f a) -> Fold s a
foldring :: (Contravariant f, Applicative f) =>
  ((a -> f a -> f a) -> f a -> s -> f a) -> LensLike f s
t a b
folded :: Foldable f => IndexedFold Int (f a) a
folded64 :: Foldable f => IndexedFold Int64 (f a) a
unfolded :: (b -> Maybe (a, b)) -> Fold b a
iterated :: Apply f => (a -> a) -> LensLike' f a a
filtered :: (Choice p, Applicative f) => (a -> Bool) ->
Optic' p f a a
filteredBy :: (Indexable i p, Applicative f) => Getting
(First i) a i -> p a (f a) -> a -> f a
backwards :: (Profunctor p, Profunctor q) => Optical p q
(Backwards f) s t a b -> Optical p q f s t a b
repeated :: Apply f => LensLike' f a a
replicated :: Int -> Fold a a
cycled :: Apply f => LensLike f s t a b -> LensLike f s
t a b
takingWhile :: (Conjoined p, Applicative f) => (a ->
Bool) -> Over p (TakingWhile p f a a) s t a a -> Over p
f s t a a
droppingWhile :: (Conjoined p, Profunctor q, Applicative
f) => (a -> Bool) -> Optical p q (Compose (State Bool)
f) s t a a -> Optical p q f s t a a
worded :: Applicative f => IndexedLensLike' Int f String
String
lined :: Applicative f => IndexedLensLike' Int f String
String


-- effect
has :: Getting Any s a -> s -> Bool
hasn't :: Getting All s a -> s -> Bool
preview :: MonadReader s m => Getting (First a) s a -> m
(Maybe a)
previews :: MonadReader s m => Getting (First r) s a ->
(a -> r) -> m (Maybe r)
preuse :: MonadState s m => Getting (First a) s a -> m
(Maybe a)
preuses :: MonadState s m => Getting (First r) s a -> (a
-> r) -> m (Maybe r)
```

```
foldMapOf :: Getting r s a -> (a -> r) -> s -> r
foldOf :: Getting a s a -> s -> a
foldrOf :: Getting (Endo r) s a -> (a -> r -> r) -> r ->
s -> r
foldlOf :: Getting (Dual (Endo r)) s a -> (r -> a -> r)
-> r -> s -> r
toListOf :: Getting (Endo [a]) s a -> s -> [a]
toNonEmptyOf :: Getting (NonEmptyDList a) s a -> s ->
NonEmpty a
anyOf :: Getting Any s a -> (a -> Bool) -> s -> Bool
allOf :: Getting All s a -> (a -> Bool) -> s -> Bool
noneOf :: Getting Any s a -> (a -> Bool) -> s -> Bool
andOf :: Getting All s Bool -> s -> Bool
orOf :: Getting Any s Bool -> s -> Bool
productOf :: Num a => Getting (Endo (Endo a)) s a -> s
-> a
sumOf :: Num a => Getting (Endo (Endo a)) s a -> s -> a
traverseOf_ :: Functor f => Getting (Traversed r f) s a
-> (a -> f r) -> s -> f ()
forOf_ :: Functor f => Getting (Traversed r f) s a -> s
-> (a -> f r) -> f ()
sequenceAOf_ :: Functor f => Getting (Traversed a f) s
(f a) -> s -> f ()
traverse1Of_ :: Functor f => Getting (TraversedF r f) s
a -> (a -> f r) -> s -> f ()
for1Of_ :: Functor f => Getting (TraversedF r f) s a ->
s -> (a -> f r) -> f ()
sequence1Of_ :: Functor f => Getting (TraversedF a f) s
(f a) -> s -> f ()
mapMOf_ :: Monad m => Getting (Sequenced r m) s a -> (a
-> m r) -> s -> m ()
forMOf_ :: Monad m => Getting (Sequenced r m) s a -> s
-> (a -> m r) -> m ()
sequenceOf_ :: Monad m => Getting (Sequenced a m) s (m
a) -> s -> m ()
asumOf :: Alternative f => Getting (Endo (f a)) s (f a)
-> s -> f a
msumOf :: MonadPlus m => Getting (Endo (m a)) s (m a) ->
s -> m a
concatMapOf :: Getting [r] s a -> (a -> [r]) -> s -> [r]
concatOf :: Getting [r] s [r] -> s -> [r]
elemOf :: Eq a => Getting Any s a -> a -> s -> Bool
notElemOf :: Eq a => Getting All s a -> a -> s -> Bool
lengthOf :: Getting (Endo (Endo Int)) s a -> s -> Int
nullOf :: Getting All s a -> s -> Bool
notNullOf :: Getting Any s a -> s -> Bool
firstOf :: Getting (Leftmost a) s a -> s -> Maybe a
first1Of :: Getting (First a) s a -> s -> a
lastOf :: Getting (Rightmost a) s a -> s -> Maybe a
last1Of :: Getting (Last a) s a -> s -> a
maximumOf :: Ord a => Getting (Endo (Endo (Maybe a))) s
a -> s -> Maybe a
maximum1Of :: Ord a => Getting (Max a) s a -> s -> a
minimumOf :: Ord a => Getting (Endo (Endo (Maybe a))) s
```

```
a -> s -> Maybe a
minimum1Of :: Ord a => Getting (Min a) s a -> s -> a
maximumByOf :: Getting (Endo (Endo (Maybe a))) s a -> (a
-> a -> Ordering) -> s -> Maybe a
minimumByOf :: Getting (Endo (Endo (Maybe a))) s a -> (a
-> a -> Ordering) -> s -> Maybe a
findOf :: Getting (Endo (Maybe a)) s a -> (a -> Bool) ->
s -> Maybe a
findMOf :: Monad m => Getting (Endo (m (Maybe a))) s a
-> (a -> m Bool) -> s -> m (Maybe a)
foldrOf' :: Getting (Dual (Endo (Endo r))) s a -> (a ->
r -> r) -> r -> s -> r
foldlOf' :: Getting (Endo (Endo r)) s a -> (r -> a -> r)
-> r -> s -> r
foldr1Of :: HasCallStack => Getting (Endo (Maybe a)) s a
-> (a -> a -> a) -> s -> a
foldl1Of :: HasCallStack => Getting (Dual (Endo (Maybe
a))) s a -> (a -> a -> a) -> s -> a
foldr1Of' :: HasCallStack => Getting (Dual (Endo (Endo
(Maybe a)))) s a -> (a -> a -> a) -> s -> a
foldl1Of' :: HasCallStack => Getting (Endo (Endo (Maybe
a))) s a -> (a -> a -> a) -> s -> a
foldrMOf :: Monad m => Getting (Dual (Endo (r -> m r)))
s a -> (a -> r -> m r) -> r -> s -> m r
foldlMOf :: Monad m => Getting (Endo (r -> m r)) s a ->
(r -> a -> m r) -> r -> s -> m r
lookupOf :: Eq k => Getting (Endo (Maybe v)) s (k, v) ->
k -> s -> Maybe v


...
```

Control.Lens.Tuple

```
class Field1 s t a b | s -> a, t -> b, s b -> t, t a ->
s where
  _1 :: Lens s t a b
class Field2 s t a b | s -> a, t -> b, s b -> t, t a ->
s where
  _2 :: Lens s t a b
```

Control.Lens.At

```
class Ixed m where
  ix :: Index m -> Traversal' m (IxValue)
             -- filter index

class Ixed m => At m where
  at :: Index m -> Lens' m (Maybe (IxValue m))
             -- find index

class Contains m where
  contains :: Index m -> Lens' m Bool
```

```
sans :: At m => Index m -> m -> m
iat :: At m => Index m -> IndexedLens (Index m) m (Maybe
(IxValue m))
ixAt :: At m => Index m -> Traversal' m (IxValue m)
iix :: Ixed m => Index m -> IndexedTraversal' (Index m)
m (IxedValue m)
icontains :: Contains m => Index m -> IndexedLens'
(Index m) m Bool

type family Index s where
  Index [a] = Int
  Index (a,b) = Int
  Index (a,b,c) = Int
  Index (e -> a) = e
  Index (Map k a) = k
  ...

type family IxValue m where
  IxValue [a] = a
  IxValue (a,b) = a
  IxValue (a,b,c) = a
  IxValue (e -> a) = a
  IxValue (Map k a) = a
  ...
```

Control.Lens.Each

```
class Each s t a b | s -> a, t -> b, s b -> t, t a -> s
where
    each :: Traversal s t a b
```

Control.Lens.Index

```
-- Indexed function
newtype Indexed i a b = Indexed { runIndexed :: i -> b
-> b }

class Conjoined p => Indexable i p where
  indexed :: p a b -> i -> a -> b

-- Indexing monoid
newtype Indexing f a = Indexing { runIndexing :: Int ->
(Int, f a) }
instance Functor (Indexing f), Applicative (Indexing f),
Semigroup (Indexing f a), Monoid (Indexing f a)

indexing :: Indexable i p => LensLike' (Indexing f) s t
a b -> Over p s t a b
withIndex :: (Indexable i p, Functor f) => p (i, s)  (f
(j, t)) -> Indexed i s (f t)
asIndex :: (Indexable i p, Contravariant f, Functor f)
=> p i (f i) -> Indexed i s (f s)
```

```
indices :: (Indexable i p, Applicative f) => (i -> Bool)
-> Optical' p (Indexed i) f a a
index :: (Indexable i p, Eq i, Applicative f) => i ->
Optical' p (Indexed i) f a a
```

Control.Lens.Prism

```
type Prism s t a b = forall p f. (Choice p, Applicative
f) => p a (f b) -> p s (f t)
type Prism' s a = Prism s s a a
type APrism s t a b = Market a b a (Identity b) ->
Market a b s (Identity t)
type APrism' s a = APrism s s a a

-- lens
prism :: (b -> t) -> (s -> Either t a) -> Prism s t a b
prism' :: (b -> s) -> (s -> Maybe a) -> Prism s s a b
_Left :: Prism (Either a c) (Either b c) a b
_Right :: Prism (Either c a) (Either c b) a b
_Just :: Prism (Maybe a) (Maybe b) a b
_Nothing :: Prism' (Maybe a) ()
_Void :: Prism s s a Void
_Show :: (Read a, Show a) => Prism' String a
only :: Eq a => a -> Prism' a ()
nearly :: a -> (a -> Bool) -> Prism' a ()

class Prefixed t where
  prefixed :: t -> Prism' t t
class Suffixed t where
  suffixed :: t -> Prism' t t

-- effect
withPrism :: APrism s t a b -> ((b -> t) -> (s -> Either
t a) -> r) -> r
clonePrism :: APrism s t a b -> Prism s t a b
outside :: Representable p => APrism s t a b -> Lens (p
t r) (p s r) (p b r) (p a r)
aside :: APrism s t a b -> Prism (e, s) (e, t) (e, a)
(e, b)
without :: APrism s t a b -> APrism u v c d -> Prism
(Either s u) (Either t v) (Either a c) (Either b d)
below :: Traversable f => APrism' s a -> Prism' (f s) (f
a)
isn't :: APrism s t a b -> s -> Bool
matching :: APrism s t a b -> s -> Either t a
matching' :: LensLike (Either a) s t a b -> s -> Either
t a
```

Control.Lens.Operator

```
-- Getter
(^.) :: s -> Getting a s a -> a
                 -- view
```

```
(^@.) :: s -> IndexedGetting i (i, a) s a -> (i, a)
                -- iview

-- Fold
(^..) :: s -> Getting (Endo [a]) s a -> [a]
                -- toListOf
(^?) :: s -> Getting (First a) s a -> Maybe a
                -- preview
(^?!) :: HasCallStack => s -> Getting (Endo a) s a -> a


-- Setter
(.~) :: ASetter s t a b -> b -> s -> t
                -- set
(%~) :: ASetter s t a b -> (a -> b) -> s -> t
                -- over
(+~) :: Num a => ASetter s t a a -> a -> s -> t
(-~) :: Num a => ASetter s t a a -> a -> s -> t
(*~) :: Num a => ASetter s t a a -> a -> s -> t
(//~) :: Fractional a => ASetter s t a a -> a -> s -> t
(^~) :: (Num a, Integral e) => ASetter s t a a -> e -> s
-> t
(^^~) :: (Fractional a, Integral e) => ASetter s t a a
-> e -> s -> t
(**~) :: Floating a => ASetter s t a a -> a -> s -> t
(||~) :: ASetter s t Bool Bool -> Bool -> s -> t
(<>~) :: Semigroup a => ASetter s t a a -> a -> s -> t
(&&~) :: ASetter s t Bool Bool -> Bool -> s -> t
(<.~) :: ASetter s t a b -> b -> s -> (b, t)
(?~) :: ASetter s t a (Maybe b) -> b -> s -> t
(<?~) :: ASetter s t a (Maybe b) -> b -> s -> (b, t)

-- Setter + index
(%@~) :: AnIndexedSetter i s t a b -> (i -> a -> b) -> s
-> t
(.@~) :: AnIndexedSetter i s t a b -> (i -> b) -> s -> t
(%@=) :: MonadState s m => AnIndexedSetter i s s a b ->
(i -> a -> b) -> m ()
(.@=) :: MonadState s m => AnIndexedSetter i s s a b ->
(i -> b) -> m ()

-- Setter + State
(.=) :: MonadState s m => ASetter s s a b -> b -> m ()
(%=) :: MonadState s m => ASetter s s a b -> (a -> b) ->
m ()
(+=) :: (MonadState s m, Num a) => ASetter' s a -> a ->
m ()
(-=) :: (MonadState s m, Num a) => ASetter' s a -> a ->
m ()
(*=) :: (MonadState s m, Num a) => ASetter' s a -> a ->
m ()
(//=) :: (MonadState s m, Fractional a) => ASetter' s a
-> a -> m ()
(^=) :: (MonadState s m, Num a, Integral e) => ASetter'
```

```
s a -> e -> m ()
(^^=) :: (MonadState s m, Fractional a, Integral e) =>
ASetter' s a -> e -> m ()
(**=) :: (MonadState s m, Floating a) => ASetter' s a ->
a -> m ()
(||=) :: MonadState s m => ASetter' s Bool -> Bool -> m
()
(<>=) :: (MonadState s m, Semigroup a) => ASetter' s a
-> a -> m ()
(&&=) :: MonadState s m => ASetter' s Bool -> Bool -> m
()
(<.=) :: MonadState s m => ASetter s s a b -> b -> m b
(?=) :: MonadState s m => ASetter s s a (Maybe b) -> b
-> m ()
(<?=) :: MonadState s m => ASetter s s a (Maybe b) -> b
-> m b
(<~) :: MonadState s m => ASetter s s a b -> m b -> m ()
```

Usage: change of variables

TODO

Usage: update value with a monadic setter (a -> m b)
Use **traverseOf**

```
-- e.g. update [a] -> [b] with (a -> m b)
value :: [a]
update :: a -> IO b
traverseOf :: LensLike f s t a b -> (a -> f b) -> s -> f
s
traverseOf each update value  :: IO [b]           --
update [a] -> [b]
```

Basics

```
-- lens :: (s -> a) -> (s -> b -> t) -> Lens s t a b
--
-- General lens constructor
--
--   len
--     (getter :: s -> a)
--     (setter:: s -> b -> t)
--
lens :: (s -> a) -> (s -> b -> t) -> Lens s t a b
lens getter setter f v = (setter v) <$> f (getter v)

-- get
(1, True) ^. (_1 . lens (*3) (const id))       -- 1 * 3
(1, True) ^. (_2 . lens not (const id))        -- False
-- e.g. conversion
(1, "A") ^. (_1 . lens (realToFrac @Int @Double) (const
round))      -- Lens' (Int, String) Double  i.e. convert
Int<->Double
```

```
(1, "A") ^. (_2 . lens (pack) (const unpack))
            -- Lens' (Int, String) Text    i.e. convert
String<->Text

-- set
(3, True) & (_1 . lens id (const id)) .~ 10        --
replace value (10, True)
(3, True) & (_2 . lens id (const id)) .~ False     --
replace value (10, True)
(3, True) & (_1 . lens id (+)) .~ 10               --
combine value (13, True)
(3, True) & (_2 . lens id (const show)) .~ False   --
(1, "False")


-- case: single value
--
1 ^. id                                 --
(1,True) ^. _1                          -- 1
(1,True) ^. _2                          -- True
-- map
1 ^. (to (*2))                          -- 2
1 ^. (lens (*2) (const (`div` 2)))      -- same with lens
1 ^. (to show)                          -- "1"
1 ^. (lens show (const read))           -- same


-- case: list
--
toListOf traverse [1,2]                             --
[1, 2]
[1,2] ^.. traverse                                  --
same
[1,2] ^.. id                                        --
same
[(1, True), (2, False)] ^.. (traverse . _1)         --
[1,2]
[(1, True), (2, False)] ^.. (traverse . _2)         --
[True,False]

-- element
[(1, True), (2, False)] ^? (element 0)              --
Just (1,True)
[(1, True), (2, False)] ^? (traverse . element 0)   --
ERROR
[(1, True), (2, False)] ^? (element 1000)           --
Nothing
[(1, True), (2, False)] ^? (element 0. _1)          --
Just 1
[1,2,3] ^.. (elements even)                         --
[1,3]  i.e. every even index

-- filter
[1,2,3,4] ^.. (traverse . filtered even)            --
[2,4] i.e. every even number
```

```
-- map
[1,2,3] ^.. (traverse . lens show (const
read))           -- ["1","2","3"]
```

Getter

```
-- view :: MonadReader s m => Getting a s a -> m a
-- (^.) :: s -> Getting a s a -> a
-- i.e. get value
--
-- Primary reader is a function
-- i.e. MonadReader r (r->)
-- e.g. m a = (Int, Bool) -> a
--
view id (1, True)         -- (1, True)     view ::
Getting  a a a -> a -> a
view _1 (1, True)         -- 1             view ::
Getting  Int (Int, Bool) Int -> (Int, Bool) -> Int
view _2 (1, True)         -- True          view ::
Getting  Bool (Int, Bool) Bool -> (Int, Bool) -> Bool

(1, True) ^. id           -- same
(1, True) ^. _1


-- to :: (Profunctor f, Contravariant f) => (s -> a) ->
Optic' p f s a
--
-- mapping getters
(1, True) ^. (_1 . to (*3))               -- 3
(1, True) ^. (_1 . lens (*3) (const id)   -- 3 same
with lens
(1, True) ^. (_1 . to not)                -- False
```

==========

======

Fold

Usage: traverse or folded to iterate
In most cases, **either** traverse or folded can be used.
folded is needed when computing with an **index**

```
 traverse :: (Traversable t, Applicative f)                                  =>     (a ->
f a) -> t a -> f (t a)
 folded   :: (Foldable t, Indexable i p, Contravariant f, Functor f)  =>     p a (f
a) -> (t a) -> f (t a)
             Foldable t                                                      =>
IndexedFold Int (t a) a
```

e.g.

```
data Alpha = A | B | C | D | E | ...
```

```
-- case: no index
-- Either folded or traverse can be used
let x = [(A, True), (C, False)]
x & preview (folded . _1)                    -- Just A
x & preview (traverse . _1)                  -- same
x & toListOF (folded . _1)                   -- [A,C]
x & toListOF (traverse . _1)                 -- same


-- case: index
-- folded only
x & ifoldrOf folded (\i (a, b) r -> (i,a):r) []
  -- OK [(0,A), (1,B), ... ]
x & ifoldrOf traverse (\i (a, b) r -> (i,a):r) []
  -- ERROR. Not indexed

x & elemIndexOf (folded . _1) A
  -- OK Just 0
x & elemIndexOf (folded . _1) C
  -- OK Just 1
x & elemIndexOf (folded . _1) Z
 -- OK Nothing
x & elemIndexOf (traverse . _1) A
  -- ERROR. Not indexed.
```

```
---- Basic
--
--
data Alpha = A | B | C | D | ...
  deriving (Eq, Ord, Show, Bounded)
instance Semigroup Alpha where (<>) = max
instance Monoid Alpha where mempty = minBound
A <> B <> C                                  -- C i.e.
takes largest

foldOf :: Getting s a -> s -> a
[A,B] & foldOf traverse                      -- fold
[A,B] = A <> B <> empty = B
Just A & foldOf traverse                     -- fold
(Just A) = A


---- Combinators
--
-- Access a Fold property
([A,B], ()) & foldOf (_1 . traverse)            -- fold
[A,B] = B
let x = (
 [
  Just [A,B],                                --
_1.element 0._Just
  Nothing                                    --
_1.element 1._Just
```

```
 ],
 [C,D]
)
x & foldOf (_1 . element 0 . _Just . traverse)    --
fold [A,B] = A<>B = B
x & foldOf (_1 . element 1 . _Just . traverse)    --
fold [] = mempty
x & foldOf (_2 . folded)                          --
fold [C,D] = C<>D = D


-- Access properties of a Fold
let x = [
 (A,[Just B,Just C]),
 (X,[Just Y, Nothing])
]
x & foldOf (traverse . _1)                        --
fold [A,X] = A<>X
x & foldOf (traverse . _2 . element 0 . _Just)    --
fold [B,Y] = B<>Y
x & foldOf (traverse . _2 . element 1 . _Just)    --
fold [C] = C
x & foldOf (traverse . _2 . element 2 . _Just)    --
fold [] = mempty


-- nested Folds
let x = [
 ([A,B], [[C,D],[E,F]]),
 ([S,T], [[U],[V,X]])
]
x & foldOf (traverse . _1 . traverse)
 -- fold [A,B,S,T] = A<>B<>S<>T = T
x & foldOf (traverse . _2 . traverse . element 0)
  -- fold [C,E,U,V] = C<>E<>U<>V = V
x & foldOf (traverse . _2 . traverse . element 1)
   -- fold [D,F,X] = D<>F<>X = X
x & foldOf (traverse . _2 . traverse . element 10)
-- fold [] = mempty


---- Fold combinators
repeated :: Apply f => a -> LensLike' f a a
replicated :: Int -> Fold a a
iterated :: Apply f => (a -> a) -> LensLike' f a a
1 & toList repeated            -- [1,1,1,..]
1 & toList iterated (*2)       -- [1,2,4,8,...]
1 & toList replicated 5        -- [1,1,1,1,1]
1 & toList (takingWhile (<3) (iterated (+1))      --
[1,2]
1 & toList (droppingWhile (<3) (iterated (+1))    --
[3,4,... ]
[1,2..] & toList (takingWhile (<3) traverse)      --
[1,2]
[1..10] & toList (traverse . filtered even)       --
```

```
[2,4,6,8,10]
... etc


---- Functor effects
--
-- pre, preview: fold to first element
preview :: MonadReader s m => Getting (First a) s a -> m
(Maybe a)
preview :: MonadReader s m => Lens' s a -> m (Maybe a)
             -- with Lens
preview :: MonadReader s m => Traversal' s a -> m (Maybe
a)           -- with Traversal
preview :: MonadReader s m => Getter' s a -> m (Maybe
a)              -- with Getter
preview :: MonadReader s m => Fold s a -> m (Maybe a)
              -- with Fold
preview :: Getting (First a) s a -> s -> Maybe a
               -- With function reader
preview :: Lens' s a -> s -> Maybe a
preview :: Traversal' s a -> s -> Maybe a
preview :: Getter' s a -> s -> Maybe a
preview :: Fold s a -> s -> Maybe a
[A,B] & preview traverse            -- Just A
[A,B] & preview (element 1)         -- Just B
[] & preview traverse               -- Nothing
[A,B] & view (pre traverse)         -- same as preview l
= view (pre l)

-- has: fold to existence
has :: Getting Any s a -> s -> Bool
has :: Lens' s a -> s -> Bool
has :: Traversal' s a -> s -> Bool
has :: Getter' s a -> s -> Bool
has :: Fold s a -> s -> Bool
Just A & has _Just               -- True
Just A & has _Nothing            -- False
(1,2) & has _1                   -- Always True
(1,2) & has _3                   -- ERROR
[A,B] & has (element 0)          -- True
[A,B] & has (element 10)         -- False

-- Prelude
[A,B] & foldOf traverse             -- fold [A,B] =
A<>B
A & foldOf id                       -- fold (Const A)
[1..10] & anyOf traverse (>0)       -- True
[1..10] & allOf traverse (>0)       -- True
[True, False] & andOf traverse      -- False
[True, False] & orOf traverse       -- True
[1..10] & findOf traverse (>0)      -- Just 1
[1..10] & lengthOf traverse         -- 10
[1..10] & nullOf traverse           -- False
... etc
```

```
-- Index + Map
--
-- at: find index, if it exists
at :: Index m -> Lens' m (Maybe IxValue m)
m :: Map String Int                    -- e.g. m = { "a":
1 }
m ^. at "a"                            -- Just 1. Gets
property "a" if it exists
view (at "a") m                        -- same

-- ix: filter index
ix :: Index m -> Traversal' m (IxValue m)
m ^? ix "a"                            -- Just 1. Gets
the first property "a" if it exists
firstOf (ix "a") m                     -- same
m ^?! ix "a"                           -- 1. Same as ^?!
but force Just
m ^?! ix "b"                           -- ERROR
m ^.. ix "a"                           -- [1]
```

==========

Prism

```
---- Basic
--
-- Composing lenses
isn't :: APrism s t a b -> s -> Bool
Just True & isn't _Just                        -- False
Nothing & isn't _Just                          -- True

-- composed
x = Just (Right (Left (True, ())))
x & isn't (_Just . _Right . _Left)             -- False
x & isn't (_Just . _Right . _Right)            -- True
x & isn't (_Just . _Right)                     -- False
x & isn't (_Just . _Left)                      -- True

-- compose with non-prism
x = Just (Right True, False)
x & isn't _Just                                -- OK.
False
x & isn't (_Just . _1)                         -- ERROR.
_1 is not a prism
x & isn't (_Just . _1 . _Right)                -- ERROR
x & preview (_Just . _1 . _Right)              -- Just
True. OK, can use with non-prism functions
x & preview (_Just . _1)                       -- Just
(Right True)
```

Example: pattern matching with outside + set

e.g. maybe and either with outside

```
import Control.Lens
import Data.Function

maybeP :: b -> (a -> b) -> Maybe a -> b
maybeP b f =
 maybeP b f
 & set (outside _Nothing) (const b)
 & set (outside _Just) f

eitherP :: (a -> r) -> (b -> r) -> Either a b -> r
eitherP f g =
  eitherP f g
  & set (outside _Left) f
  & set (outside _Right) g


-- e.g.
maybeP 0 (+10) (Just 1)               -- 11
maybe 0 (+10) (Just 1)                -- same
maybeP 0 (+10) Nothing                -- 0
eitherP (+1) (+100) (Left 1)          -- 2
eitherP (+1) (+100) (Right 1)         -- 101
```

==========

Template Haskell

Control.Lens.TH: https://hackage.haskell.org/package/lens/docs/Control-Lens-TH.htm

```
-- Control.Lens.TH
makeLenses :: Name -> Q Dec
makeLensesFor :: [(String, String)] -> Q Dec
makeLensesWith :: LensRules -> Name -> Q Dec
makePrisms :: Name -> Q Dec
...
```

Lens rules

```
-- Control.Lens.TH
data LensRules
lensField :: Lens' LensRules FieldNamer
simpleLenses :: Lens' LensRules Bool
createClass :: Lens' LensRules Bool
generateSignatures :: Lens' LensRules Bool
generateLazyPatters :: Lens' LensRules Bool

-- defaults
lensRules :: LensRules
```

```
lensRulesFor :: [(String, String)] -> LensRules
classyRules :: LensRules
defaultFieldRules :: LensRules
camelCaseFieldRules :: LensRules
underscoreFields :: LensRules
abbreviatedFields :: LensRules

-- naming
type FieldNamer :: Name -> [Name] -> Name -> [DefName]
type ClassyName :: Name -> Maybe (Name Name)
data DefName =
    TopName Name
  | MethodName Name Name

mappingNamer :: (String -> [String]) -> FieldNamer
camelCaseNamer :: FieldNamer
underscoreNamer :: FieldNamer
underscoreNoPrefixNamer :: FieldNamer
classUnderscoreNoPrefixNamer :: FieldNamer
abbreviatedNamer :: FieldNamer
```

Example: generate field lenses and constructor prisms
By default, fields need to be prefixed with **underscore**

```
{-# LANGUAGE TemplateHaskell #-}
import Control.Lens
import Control.Lens.TH

data MyData a = MyData { _a :: a, _b :: Int }

data MyMaybe a = MyJust a | MyNothing

-- Generate field lenses
$(makeLenses ''MyData)                                  --
creates
                                                        --
     a :: Lens' (MyData a) a
                                                        --
    b :: Lens' MyData Int

-- Generate prism
$(makePrisms ''MyMaybe)                                 --
creates
                                                        --
     _MyJust :: Prism' (MyMaybe a) a
                                                        --
    _MyNothing :: Prism' (MyMaybe a) ()


-- usage
view a (MyData "hello" 1)           -- "hello"
view b (MyData "hello" 1)           -- 1
```

```
matching _MyJust (MyJust 1)         -- Right 1
matching _MyJust MyNothing          -- Left MyNothing
matching _MyNothing (MyJust 1)      -- Left (MyJust 1)
matching _MyNothing MyNothing       -- Right ()
```

Equivalent defined manually

```
{-# LANGUAGE LambdaCase #-}

import Control.Lens

-- Lens
a :: Lens' (MyData a) a
a l myData = (\a' -> myData {_a = a'}) <$> l (_a myData)

b :: Lens' (MyData a) String
b l myData = (\b' -> myData {_b = b'}) <$> l (_b myData)


-- Prism
_MyJust :: Prism' (MyMaybe a) a
_MyJust l =
  dimap
    ( \case
        MyJust a -> Left a
        MyNothing -> Right MyNothing
    )
    ( \case
        Left a' -> MyJust <$> a'
        Right s -> pure s
    )
    (left' l)

_MyNothing :: Prism' (MyMaybe a) ()
_MyNothing l =
  dimap
    ( \case
        MyNothing -> Left ()
        v@(MyJust _) -> Right v
    )
    ( \case
        Left x -> MyNothing <$ x                --
IMPORTANT: Not (pure MyNothing :: f (MyMaybe a) but
(MyNothing <$ x :: f (MyMaybe a)) to preserve effect of
f
        Right s -> pure s
    )
    (left' l)
```

Example: customize field lens names
Use a custom **FieldNamer** + **makeLensesWith**

e.g.

```
module Utils (                                          --
IMPORTANT: rules need to be defined in separate module
due to template haskell restriction
 dataLensRules
) where

import Control.Lens.TH

lensNamer :: FieldNamer
lensNamer = mappingNamer $ \s -> ['_' : s]            --
prefix each lens name with '_'

dataLensRules :: LensRules
dataLensRules =
  lensRules
    & set lensField lensNamer
```

```
{-# LANGUAGE TemplateHaskell #-}
import Control.Lens
import Control.Lens.TH
import Utils

data MyData a = MyData { a :: a, b :: Int }

$(makeLensesWith dataLensRules ''MyData)            --
creates
                                                    --
  _a :: Lens' (MyData a) a
                                                    --
   _b :: Lens' (MyData a) Int

-- e.g.
x = view _a (MyData "hello" 1)       -- "hello"
x2 = view _b (MyData "hello" 1)      -- 1
```

==========
Data.Aeson

https://hackage.haskell.org/package/lens-aeson

```
-- Data.Aeson.Lens

class AsNumber t where
 _Integer :: Prism' t Scientific
 _Double :: Prism' t Double
 _Integer :: Prism' t Integer
```

```
_Integeral :: (AsNumber t, Integral a) => Prism' t a

class AsNumber t => AsValue t where
 _Value :: Prism' t Value
 _String :: Prism' t String
 _Null :: Prism' t ()
 _Bool :: Prism' t Bool
 _Array :: Prism' t (Vector Value)
 _Object :: Prism' t (KeyMap Value)

class IsKey t where
 _Key :: Iso' t y

class AsJSON t where
 _JSON :: (FromJSON a, ToJSON b) => Prism t t a b


nonNull :: Prism Value Value
key :: AsValue t => Key -> Traversal' t Value
                 -- _Object . ix <key>
atKey :: AsValue t => Key -> Traversal' t (Maybe
value)                    -- _Object . at <key>
members :: AsValue t => IndexedTraversal' Key t Value
                 -- _Object . traversed
nth :: AsValue t => Int -> Traversal' t Value
                 -- _Array . ix <int>
values :: AsValue t => IndexedTraversal' Int t Value
                 -- _Array . traversed
```

```
---- Basic
--
x :: Value
Null ^? _Null                 == Just ()
String "a" ^? _Null           == Nothing

String "a" ^? _String         == Just ("a" :: Text)
Number 1 ^? _Number           == Just (1 :: Scientific)
Number 1 ^? _Integer          == Just (1 :: Integer)
Number 1 ^? _Double           == Just (1 :: Double)

-- mapping
x ^? (_Number . lens (*2) (const id))
         -- map Number e.g.  x * 2
x ^? (_String . lens ("Hello") (const id))
         -- map String e.g. "Hello abc" if x="abc"
x ^.. (values . _Number . lens (*2) (const id))
         -- map Array  e.g. [2,4,6] if x=[1,2,3]
-- e.g. conversion
x ^? (_String . lens (Text.unpack) (const Text.pack))
         -- Traversal' Value String
x ^? (_Number . lens (round) (const (realToFrac
```

```
@Integer)))      -- Traversal' Value Integer



-- Object
--
-- e.g. x = {"name":"A","value":1}
x ^? key "name"                              == Just
(String "A")
x ^? _Object . ix "name"
             -- same
x ^? key "name" . _String                    == Just "A"
x ^? key "name" . _Integer                   == Nothing

x ^? atKey "name"                            == Just (Just
"A")
x ^? _Object . at "name"
             -- same
x ^? atKey "name" . _Just . _String          == Just "A"

-- update
(key "name" .~ String "a") x                 -- update
value {"name":"a","value":1}
(key "name" . _String .~ "a") x              -- same
(key "abc" . _Bool .~ True) x                -- BAD.
Nothing happens if key does not exist
(atKey "abc" . _Bool .~ Just True) x         -- add new
key {"abc":true,"name":"a","value":1}
(atKey "abc" . _Bool ?~ True) x              -- same
(atKey "name" .~ Nothing) x                  -- delete key
{"value": 1}



-- Array
--
-- e.g. x = [
--  {"name": "A", "value": 1},
--  {"name": "B", "value": 2},
-- ]
x ^. _Array                                  -- Vector
x ^.. values                                 -- [Object
{name:"A",value:"1"}, ...]
x ^.. (_Array . each)                        -- same
x ^.. (values . key "name")                  -- [String
"A", String "B"]
x ^.. (values . key "name" . _String)        -- ["A", "B"]
x ^.. (values . key "name" . _Integer)       -- []

-- index
x ^? (nth 0)                                 -- Just
(Object {name:"A",value:1})
x ^? (nth 10000)                             -- Nothing
```

```
-- filter
hasValue :: (Eq a) => Traversal' Value a -> a -> Value
-> Bool
hasValue l v = (== Just v) . preview l
x ^? (values .filtered (hasValue (key "name" . _String)
"A"))        -- Just (Object {name:"A",value:1})
x ^? (values .filtered (hasValue (key "name" . _String)
"X"))        -- Nothing


-- array to list
--
-- e.g. if x=[1,2,3]
{-# LANGUAGE OverloadedStrings #-}
import GHC.Exts (fromList, toList)
arrayToList :: Traversal' (Value) [Value]
arrayToList = _Array . lens toList (const fromList)
x ^.. (arrayToList . traverse . _Number)
                  -- [1,2,3]
x ^.. (arrayToList . traverse . _Number . lens (* 1000)
(const id))       -- [1000,200,3000]


-- add/remove items
import Data.Vector qualified as V
(_Array %~ `V.snoc` String "Hello") x
              -- Append i.e. x ++ [String "Hello"]
(_Array %~ `V.filter` (hasValue (key "name" . _String)
"A"))) x      -- Remove i.e. [Object {name:"A",value:1}]

-- update inner
((values . key "name" . _String) .~ "X")
x                 -- update all items [Object
{name:"X",value:1}, Object {name:"X",value:1}]
((nth 0 . key "name" . _String) .~ "X")
x                 -- update first item
((nth 10000 . key "name" . _String) .~ "X")
x             -- no change
((filtered ... . key "name" . _String) .~ "X")
x        -- update filtered item
```